

Derivatives Pricing Engine

*A Complete Learning Guide — Theory, Architecture, and Code
(Options on Futures, Black-76)*

Covers: Black-76, binomial trees (CRR & Leisen-Reimer), Monte Carlo
(Asian / Lookback / Basket), Crank-Nicolson barrier PDE, Greeks,
scenario analysis, visualization, and the Streamlit dashboard.

Generated learning document for the pricing_engine project

Table of Contents

Table of Contents.....	2
1 Project Overview.....	5
1.1 What the Project Does.....	5
1.2 The Problem It Solves.....	5
1.3 Why This Is Useful.....	5
1.4 Overall Architecture.....	6
1.5 End-to-End Workflow.....	6
2 Theory.....	8
2.1 Futures Prices and the Driftless Martingale.....	8
2.2 Geometric Brownian Motion (GBM).....	8
2.3 The Black-76 PDE and Closed-Form Price.....	9
2.4 The Greeks.....	9
2.5 Implied Volatility and Newton-Raphson.....	10
2.6 Binomial Trees: Cox-Ross-Rubinstein (CRR).....	10
2.7 Leisen-Reimer Trees and the Peizer-Pratt Inversion.....	11
2.8 Monte Carlo Simulation.....	11
2.8.1 Antithetic Variates (variance reduction).....	12
2.8.2 Confidence Intervals.....	12
2.9 Path-Dependent Payoffs: Asian and Lookback Options.....	12
2.9.1 The Geometric Asian Closed Form.....	12
2.10 Basket Options and Cholesky Correlation.....	13
2.11 Finite-Difference PDE Methods: Crank-Nicolson.....	13
2.12 In-Out Parity for Barrier Options.....	14
2.13 Scenario Analysis and the Greeks-Taylor Connection.....	14
3 Codebase Structure.....	15
4 Code Walkthrough.....	17
4.1 models/base.py — The Shared Interface.....	17
4.2 models/black76.py — The Analytic Pricer.....	18
4.3 calibration/implicit_vol.py — Solving for σ	18
4.4 models/crr.py — The CRR Binomial Tree.....	18
4.5 models/leisen_reimer.py — The Leisen-Reimer Tree.....	19

4.6 models/monte_carlo.py — Simulation-Based Pricers	19
4.7 models/barrier_pde.py — Crank-Nicolson PDE Solver	20
4.8 greeks/engine.py — The Greeks Aggregator	21
4.9 scenario_analysis/engine.py — The Shock Grid	21
4.10 visualization/plots.py — Matplotlib Charts	22
4.11 dashboard/app.py — The Streamlit UI	22
5 Theory → Code Connection	24
5.1 Black-76 Closed Form.....	24
5.2 Newton-Raphson Implied Volatility	24
5.3 CRR Binomial Tree.....	24
5.4 Peizer-Pratt / Leisen-Reimer	24
5.5 Exact GBM Simulation + Antithetic Variates	25
5.6 Cholesky-Correlated Basket Simulation	25
5.7 Crank-Nicolson Barrier PDE.....	25
5.8 In-Out Parity.....	25
5.9 Finite-Difference Greeks	26
6 Complete Execution Flow.....	27
6.1 python -m pricing_engine.validate_loz6.....	27
6.2 streamlit run pricing_engine/dashboard/app.py.....	27
6.3 python -m pricing_engine.generate_loz6_figures	27
7 Step-by-Step Example	28
7.1 Initial Input.....	28
7.2 Black-76 Analytic Price	28
7.3 Greeks (analytic)	28
7.4 Cross-Checking with CRR and Leisen-Reimer.....	28
7.5 Cross-Checking with Monte Carlo.....	29
7.6 Scenario Shock: F +1%.....	29
8 Important Design Decisions	30
8.1 Why an Abstract Base Class + Dataclasses.....	30
8.2 Why Finite-Difference Greeks by Default	30
8.3 Why a Fixed Random Seed as a Dataclass Field.....	30
8.4 Why In-Out Parity Instead of a Second PDE Solve	30
8.5 Library Choices	31

8.6 Known Limitations.....	31
8.7 Possible Improvements.....	32
9 Learning Notes (Consolidated).....	33
9.1 One Formula per Section	33
9.2 One Function/Class per Module	33
9.3 Common Mistakes, Collected	33
9.4 What You Must Understand Before Modifying This Codebase	34
10 Final Cheat Sheet.....	35
10.1 Architecture in One Picture	35
10.2 Key Concepts.....	35
10.3 Formula Quick-Reference	35
10.4 Important Functions / Classes.....	36
10.5 Execution Flow (Any Entry Point).....	36
10.6 Inputs / Outputs Summary.....	36
10.7 Assumptions.....	36
10.8 Limitations.....	37

1 Project Overview

1.1 What the Project Does

This project is a **modular Python engine that prices options on futures** (the Black-76 world, used for commodities, rates futures, and index futures), computes their risk sensitivities (the **Greeks**), stress-tests them under market shocks, visualizes the results, and exposes everything through an interactive **Streamlit dashboard**.

- **6 pricing engines** that all agree with each other: an analytic closed form (Black-76), two binomial trees (CRR, Leisen-Reimer), a Monte Carlo simulator (Asian / Lookback / Basket options), and a finite-difference PDE solver (barrier options).
- **1 shared interface** (`OptionModel`) so every model automatically supports the same `.price()`, `.delta()`, `.gamma()`, `.vega()`, `.theta()`, `.rho()` calls.
- **Analytics layers** built once, on top of that interface: a Greeks aggregator, a scenario/shock engine, chart generation, and a live dashboard.

1.2 The Problem It Solves

A trading or risk desk never prices an option with just one method. A vanilla European option has a fast closed form; an American option needs a tree; a path-dependent option (Asian, lookback, basket) needs Monte Carlo; a barrier option is naturally suited to a PDE grid. If each of those is built as an isolated script, three problems appear:

1. ****Inconsistent risk****: every model would need its own hand-written Greeks code, which is easy to get subtly wrong and hard to keep consistent across models.
2. ****No cross-checking****: without a common interface, it is hard to verify that a tree and a Monte Carlo simulation actually agree with the analytic price.
3. ****Duplicated plumbing****: scenario analysis, reporting, and visualization would have to be re-written per model instead of written once.

This project solves that by making every pricer a subclass of one abstract base class. Any function written against that base class (Greeks, scenario shocks, plotting, the dashboard) works for **every** model without modification.

1.3 Why This Is Useful

- **Cross-validation is built in**: Phase-by-phase, every new pricer was tested against Black-76 (the analytic ground truth) before being trusted.
- **Adding a 7th model is cheap**: implement `.price()`, inherit Greeks for free, and it instantly works in the Greeks table, the scenario engine, and the dashboard.
- **It is a compact, working syllabus** for quantitative finance: martingale pricing, PDEs, trees, Monte Carlo, and risk management, all cross-referenced against each other with real numbers.

1.4 Overall Architecture

The codebase is organized into four layers. Lower layers know nothing about higher layers — `models/`` has no idea a dashboard exists — which is what keeps the dependency graph simple and testable.

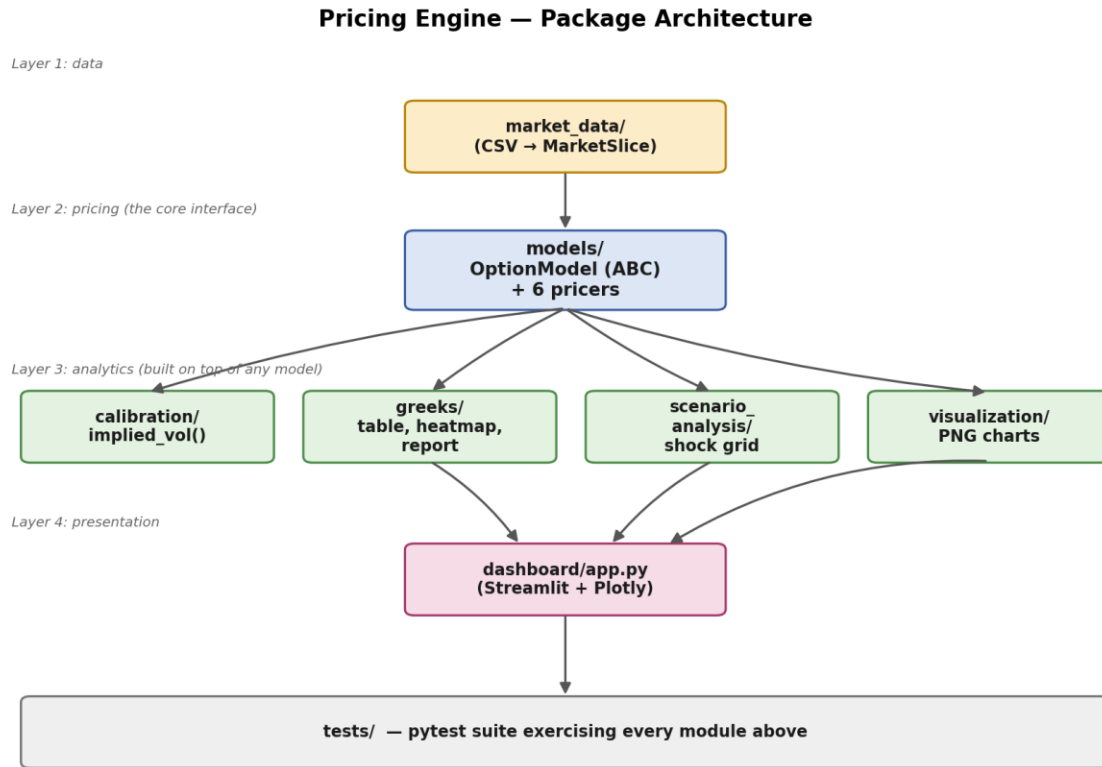


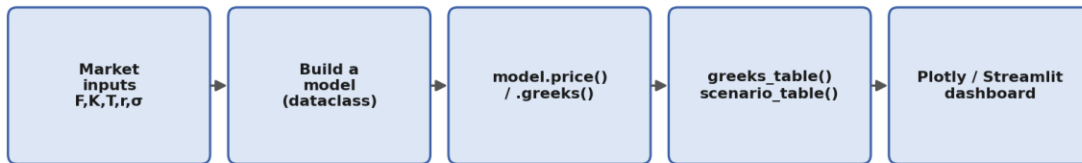
Figure 1.1 — Package architecture: four layers, one shared model interface.

Layer	Package(s)	Responsibility
1 — Data	market_data/	Read a CSV of exchange settlements into typed Python objects.
2 — Pricing	models/	The OptionModel interface + 6 concrete pricers. The core of the project.
3 — Analytics	calibration/, greeks/, scenario_analysis/, visualization/	Generic tools that operate on <i>*any*</i> OptionModel instance.
4 — Presentation	dashboard/	A Streamlit + Plotly UI wiring the layers above to widgets.

1.5 End-to-End Workflow

Concretely, running the project looks like this, regardless of which model is used:

End-to-End Data Flow (one instrument)



Every arrow above is a plain Python function/method call — no hidden global state.

Figure 1.2 — Data flow for one instrument, from market inputs to dashboard.

4. **Market inputs** arrive either as literal numbers (F, K, T, r, σ) or are read from a settlement CSV via `market_data.load_slice()`.
5. **A model is built**: one line, e.g. `Black76(F=93.9, K=94, T=0.186, r=0.042, sigma=0.506, option_type="call")`.
6. **The model is queried**: `.price()` and `.greeks()` are the two calls every downstream tool relies on.
7. **Analytics are layered on**: `greeks_table([model, ...])` and `scenario_table(model)` reprice the model many times under bumped inputs.
8. **Results are shown**: as a printed table (`validate_loz6.py`), as PNG charts (`visualization/`), or live in the Streamlit dashboard.

🔑 Section 1 — What You Must Remember

- The project is a **pricing engine**, not a single script: one interface, six pricers, four analytics tools, one UI.
- `OptionModel` is the seam that makes the whole system composable — every other module depends on it, nothing depends on them.
- Every model must implement `price()`. Everything else (Greeks, scenario shocks, charts) is generic code layered on top.
- Common mistake: assuming each pricer has its own bespoke Greeks formulas — most rely on the shared finite-difference default in `base.py`.

2 Theory

This section builds up every mathematical idea used in the code, in the order the project itself was built. Each subsection ends with a short worked numerical example so the formula is never just abstract symbols.

2.1 Futures Prices and the Driftless Martingale

Definition. A futures contract obligates you to buy/sell an asset at a future date at a price agreed today. Unlike a stock, a futures price F costs nothing to enter into — there is no upfront investment, so, under the risk-neutral pricing measure, it cannot have any expected drift. If it did, one side of the contract could earn risk-free profit for free.

$$\mathbb{E}[dF] = 0 \Leftrightarrow dF = \sigma \cdot F \cdot dW$$

F is a driftless martingale under the risk-neutral measure

Why it matters here. This single fact is why the project calls itself a "Black-76" engine (after Fischer Black's 1976 paper on commodity futures options): every model in ``models/`` reuses the same driftless assumption. Contrast with equities, where $dS = (r-q)S dt + \sigma S dW$ has a drift term (the risk-free rate minus dividends) — futures options never need that term.

Connection to code: ``models/black76.py``, ``models/crr.py``, ``models/leisen_reimer.py``, and ``models/monte_carlo.py`` all skip the drift term that a stock-option formula would otherwise need.

2.2 Geometric Brownian Motion (GBM)

Definition. GBM is the standard model for a price that can never go negative and whose *percentage* moves (not absolute moves) are normally distributed. For the driftless futures case:

$$dF = \sigma F dW \Leftrightarrow d(\ln F) = -\frac{1}{2}\sigma^2 dt + \sigma dW$$

Intuition. Write $\ln F$ instead of F . Itô's lemma (the chain rule for stochastic calculus) says that when you transform a GBM by taking its logarithm, you get an ordinary Brownian motion with a small, constant downward drift of $-\frac{1}{2}\sigma^2$ — this is the *convexity correction*: because F is symmetric but $\ln F$ is concave, Jensen's inequality means the log-price drifts down even though F itself has no drift.

Exact simulation formula. Because σ is constant, the SDE has a closed-form solution — no discretization error is needed to simulate it:

$$F(t+\Delta t) = F(t) \cdot \exp(-\frac{1}{2}\sigma^2\Delta t + \sigma\sqrt{\Delta t} \cdot Z), \quad Z \sim N(0,1)$$

Numerical example. $F_0=100$, $\sigma=0.25$, $\Delta t=1/365$ (one day), and a draw $Z=0.40$:

```
drift = -0.5 * 0.25**2 * (1/365)      # = -0.000856
shock = 0.25 * (1/365)**0.5 * 0.40      # = +0.00526
F_next = 100 * exp(drift + shock)        # = 100.526
```

Connects to code: this exact formula is ``models/monte_carlo.py``'s `_MonteCarloOption._simulate_paths()` and ``BasketOption.price()`` — see §5.

2.3 The Black-76 PDE and Closed-Form Price

Why a PDE at all? Any price that only depends on (F, t) and follows GBM must, by a no-arbitrage argument (Itô's lemma applied to the hedged portfolio), satisfy one partial differential equation. Solve that PDE with the right terminal condition (the payoff) and you have the price.

$$\partial V / \partial t + \frac{1}{2} \sigma^2 F^2 \cdot \partial^2 V / \partial F^2 - rV = 0$$

The Black-76 PDE (no first-derivative/drift term)

Because F is driftless, there is **no** $\partial V / \partial F$ term — contrast this with the Black-Scholes-on-a-stock PDE, which has an extra $rS \cdot \partial V / \partial S$ term. This is the one PDE every model in this project ultimately answers to, whether by an exact formula, a tree, simulation, or a numerical grid.

Closed-form solution (for a European call/put, solved analytically):

$$d_1 = [\ln(F/K) + \frac{1}{2} \sigma^2 T] / (\sigma \sqrt{T}) \quad d_2 = d_1 - \sigma \sqrt{T}$$

$$\text{Call} = e^{(-rT)} \cdot [F \cdot N(d_1) - K \cdot N(d_2)] \quad \text{Put} = e^{(-rT)} \cdot [K \cdot N(-d_2) - F \cdot N(-d_1)]$$

Symbol	Meaning
F	Current futures price
K	Strike price
T	Time to expiry, in years
r	Continuously-compounded risk-free rate
σ	Volatility (annualized, decimal)
N(x)	Standard normal cumulative distribution function

Numerical example (used throughout this guide): $F=100$, $K=100$, $T=1$, $r=5\%$, $\sigma=25\%$:

```
d1 = (log(100/100) + 0.5*0.25**2*1) / (0.25*1**0.5) # = 0.125
d2 = d1 - 0.25 # = -0.125
call = exp(-0.05*1) * (100*N(0.125) - 100*N(-0.125))
      = 0.9512 * (100*0.5497 - 100*0.4503) = 9.4625
```

Connects to code: `models/black76.py` — see §4.2 and §5 for the exact line-by-line mapping.

2.4 The Greeks

Definition. The Greeks are the partial derivatives of price with respect to each input — they answer "if this one input moves a little, how much does the price move?", which is exactly what a trader needs to hedge a position.

Greek	Derivative	Answers
Delta (Δ)	$\partial V / \partial F$	How much does price move per \$1 move in F?
Gamma (Γ)	$\partial^2 V / \partial F^2$	How much does delta itself change per \$1 move in F?
Vega	$\partial V / \partial \sigma$	Sensitivity to a 1.0 (100 vol-point) change in volatility.
Theta (Θ)	$-\partial V / \partial T$	Time decay: how much value is lost per year of time passing.

Rho (ρ)	$\partial V / \partial r$	Sensitivity to a 1.0 (100%) change in the risk-free rate.
----------------	---------------------------	---

Two ways to compute them appear in this project:

9. **Analytic** (`Black76`): closed-form derivatives of the d_1/d_2 formula above. Fast and exact.
10. **Finite difference** (every other model, inherited from `OptionModel`): reprice with a tiny bump up and down, and divide by the bump size. Works for *any* pricer, even ones with no closed form, at the cost of a few extra reprices.

$$\Delta \approx [V(F+h) - V(F-h)] / (2h) \quad \Gamma \approx [V(F+h) - 2V(F) + V(F-h)] / h^2$$

Central finite differences

Numerical example (central FD delta, Black-76 call, $h=0.01$, same params as §2.3): $V(100.01)=9.4677$, $V(99.99)=9.4573 \rightarrow \Delta \approx (9.4677-9.4573)/0.02 = 0.5230$, matching the analytic delta almost exactly.

Connects to code: `models/base.py`'s `delta()/gamma()/vega()/theta()/rho()` — see §4.1.

2.5 Implied Volatility and Newton-Raphson

Definition. Implied volatility is the σ that, plugged into the pricing formula, reproduces an observed market price. It is not directly observable — it must be *solved for* by inverting the pricing formula.

Why Newton-Raphson. The pricing formula is monotonically increasing in σ , and its derivative with respect to σ is exactly **vega** — which the project already computes. That makes Newton-Raphson a natural fit: start from a guess, use vega as the local slope, and step toward the root.

$$\sigma_{n+1} = \sigma_n - [V(\sigma_n) - \text{market_price}] / \text{vega}(\sigma_n)$$

Newton-Raphson update

Why also bisection. Deep in/out-of-the-money options have tiny vega, so Newton-Raphson's step can blow up or stall. The code falls back to bisection (halving a bracket `[1e-6, 5.0]` until the price matches) — slower but guaranteed to converge as long as the market price is bracketed.

Numerical example: market price 9.4625, start $\sigma_0=0.20$. Vega at $\sigma=0.20$ is ≈ 35.4 , price at $\sigma=0.20$ is $\approx 7.97 \rightarrow \sigma_1 = 0.20 - (7.97-9.4625)/35.4 \approx 0.242$. A couple more iterations converge to $\sigma=0.25$, recovering the true input.

Connects to code: `calibration/implied_vol.py` — see §4.3.

2.6 Binomial Trees: Cox-Ross-Rubinstein (CRR)

Definition. A binomial tree approximates continuous GBM with a discrete process: over each small step Δt , the price either multiplies by u (up) or d (down). With enough steps, the tree's terminal distribution converges to the same lognormal distribution GBM implies.

Why trees at all, if Black-76 has a closed form? Because a closed form assumes European exercise (only at expiry). American options may be exercised early, which has no simple closed form — but a tree can check, *at every single node*, whether early exercise beats holding.

CRR parametrization, specialized to the driftless futures case (no growth term in p , unlike a stock tree):

$$u = e^{(\sigma\sqrt{\Delta t})} \quad d = 1/u \quad p = (1-d)/(u-d) \quad \text{discount factor} = e^{(-r\Delta t)}$$

Algorithm (backward induction): build terminal payoffs at every leaf, then work backward, at each node computing the discounted expectation over the two children, and — for American exercise — taking the max against immediate intrinsic value.

$$V(\text{node}) = \max(e^{(-r\Delta t)} \cdot [p \cdot V(\text{up}) + (1-p) \cdot V(\text{down})], \text{intrinsic}(\text{node}))$$

only the max(...) part is American; European drops it

Tiny worked example (2 steps, $F=100$, $K=100$, $\sigma=0.30$, $r=0$, $T=1$, $\Delta t=0.5$, European call):

$u=e^{(0.3\cdot\sqrt{0.5})}=1.2337$, $d=1/u=0.8106$, $p=(1-0.8106)/(1.2337-0.8106)=0.4479$. Terminal nodes: $F\cdot u^2=152.2$ (payoff 52.2), $F=100$ (payoff 0), $F\cdot d^2=65.7$ (payoff 0). Step back one level using p , then discount — this is exactly what the code's nested loop does, just for hundreds of steps instead of 2.

Connects to code: ``models/crr.py`` — see §4.4.

2.7 Leisen-Reimer Trees and the Peizer-Pratt Inversion

The problem with CRR. CRR's error oscillates and only shrinks like $O(1/n)$ — doubling the steps roughly halves the error, but the error's *sign* flips unpredictably near the strike (a well-known artifact of matching the tree to the diffusion via $\sqrt{\Delta t}$ alone).

The Leisen-Reimer (LR) fix. Instead of deriving u , d , p from $\sqrt{\Delta t}$, LR works backward from the answer: it picks p so the tree's *terminal* probability of finishing in-the-money matches the true Black-76 $N(d_2)$ probability by construction, using the **Peizer-Pratt inversion** — a classical approximation for inverting a binomial CDF into a normal one.

$$h(z, n) = \frac{1}{2} + \text{sign}(z) \cdot \sqrt{\left[\frac{1}{4} - \frac{1}{4} \cdot \exp\left(-\left(\frac{z}{(n+1/3 + 0.1/(n+1))} \right)^2 \cdot (n+1/6) \right) \right]}$$

Peizer-Pratt inversion

$$p = h(d_2, n) \quad p' = h(d_1, n) \quad u = p'/p \quad d = (1-p')/(1-p)$$

Why this converges faster. Because the tree is anchored to the exact d_1/d_2 target by construction, the oscillation cancels out and the error shrinks like $O(1/n^2)$ — roughly n times faster than CRR for the same step count. This is exactly what §7's example demonstrates numerically.

Why n must be odd. An odd number of steps keeps the middle layer of the tree centered exactly on the strike, avoiding an extra bias term.

Connects to code: ``models/leisen_reimer.py`` — see §4.5.

2.8 Monte Carlo Simulation

Definition. Instead of solving for the price analytically, simulate thousands of random possible futures for F , compute the payoff on each, and average. The Law of Large Numbers guarantees the average converges to the true expected (discounted) payoff as the number of paths grows.

$$\text{price} = e^{(-rT)} \cdot (1/N) \cdot \sum_i \text{payoff}(\text{path}_i)$$

Why it is needed here. Path-dependent payoffs (an Asian option's running average, a lookback's running min/max, a basket's correlated assets) generally have no closed form — simulating the whole path is the most general tool available.

2.8.1 Antithetic Variates (variance reduction)

Idea. For every random draw `Z`, also simulate the mirror-image path using `-Z`. Averaging a path and its mirror cancels out some of the sampling noise for free — no extra randomness is spent, just reused more cleverly. It works best when the payoff is roughly monotonic in `Z`.

2.8.2 Confidence Intervals

Why report one. A Monte Carlo price is a statistical *estimate*, not an exact answer — reporting it without an error bar is misleading. The 95% CI is built from the sample standard error of the discounted payoffs:

$$CI = \text{mean} \pm 1.96 \cdot (\text{std} / \sqrt{N})$$

1.96 is the 97.5th percentile of the standard normal

Numerical example: 100,000 discounted payoffs with mean 9.45 and sample std 14.0 → standard error = 14.0/√100000 ≈ 0.0443 → 95% CI ≈ [9.36, 9.54].

Connects to code: `models/monte\_carlo.py` — see §4.6.

2.9 Path-Dependent Payoffs: Asian and Lookback Options

Option	Payoff (call)	Intuition
Asian (arithmetic)	$\max(\text{mean}(F_t) - K, 0)$	Averaging smooths out single-day price spikes — cheaper, less volatile payoff.
Asian (geometric)	$\max(\text{geo-mean}(F_t) - K, 0)$	Same idea, but the geometric mean has a tractable closed form (§2.9.1).
Lookback (floating strike)	$F_T - \min(F_t)$	"Buy at the best possible price, sell at expiry" — always ≥ 0, so no $\max(\cdot, 0)$ is needed.

2.9.1 The Geometric Asian Closed Form

Why it exists. The geometric average of a lognormal process is itself lognormal (logs turn products into sums, and sums of normals are normal), so a geometric Asian option — unlike an arithmetic one — has an exact closed form. It is priced as a Black-76 option with an *adjusted* forward and volatility:

$$F_{\text{eff}} = F \cdot \exp(\text{mean_shift} + \frac{1}{2} \cdot \text{var_ln}) \quad \sigma_{\text{eff}} = \sqrt{(\text{var_ln} / T)}$$

as n (monitoring points) → ∞: $F_{\text{eff}} \rightarrow F \cdot e^{(-\sigma^2 T / 12)}$, $\sigma_{\text{eff}} \rightarrow \sigma / \sqrt{3}$

Why it matters for the project: this formula is the independent "answer key" used to validate the Monte Carlo Asian option in `tests/test\_monte\_carlo.py` — see §5 and §7.

2.10 Basket Options and Cholesky Correlation

Definition. A basket option pays off on a *weighted average* of several correlated underlyings, e.g. a call on $0.5 \cdot F_1 + 0.5 \cdot F_2$.

The correlation problem. Simulating each asset's random draw independently would ignore that they move together. The fix is the **Cholesky decomposition**: factor the target correlation matrix $\Sigma = L \cdot L^t$, then transform independent draws Z into correlated draws $L \cdot Z$. If Z has identity covariance, $L \cdot Z$ has covariance $L \cdot L^t = \Sigma$ exactly.

$$\Sigma = L \cdot L^t \text{ (} L \text{ lower-triangular)} \quad Z_{\text{correlated}} = L \cdot Z_{\text{independent}}$$

Cholesky factorization

Numerical example (2 assets, correlation $\rho=0.5$): $L = \begin{bmatrix} 1 & 0 \\ 0.5 & 0.866 \end{bmatrix}$. An independent draw $Z=(1, 0)$ becomes $Z_{\text{corr}}=(1, 0.5)$ — asset 2's shock is now partly explained by asset 1's.

Connects to code: `models/monte_carlo.py`'s `BasketOption.price()` calls `numpy.linalg.cholesky()` directly — see §4.6.

2.11 Finite-Difference PDE Methods: Crank-Nicolson

Why a PDE grid for barriers. A barrier option's boundary condition (price = 0 the instant the underlying touches a level) is naturally expressed as an edge of a numerical grid — something trees and closed forms cannot represent directly.

Substitution. Let $\tau = T - t$ (time-to-expiry, counting up from 0 at expiry). This turns the backward-in-time Black-76 PDE into a forward-in- τ diffusion — the same direction ordinary heat-equation solvers expect:

$$\partial V / \partial \tau = \frac{1}{2} \sigma^2 F^2 \cdot \partial^2 V / \partial F^2 - rV, \text{ stepped forward from } \tau=0 \text{ (payoff) to } \tau=T \text{ (today)}$$

Crank-Nicolson averages the *explicit* method (use only the known, old time layer — fast but unstable for large steps) and the *implicit* method (solve for the new layer using only new-layer values — stable but needs a linear solve) 50/50. This gives second-order accuracy in both space and time and is unconditionally stable.

$$(V_i^{n+1} - V_i^n) / \Delta \tau = \frac{1}{2} \cdot [RHS(\text{new layer}) + RHS(\text{old layer})]$$

Rearranged, each time step becomes a **tridiagonal linear system** (each row only touches its immediate neighbours) — solvable in $O(N)$ per step via the **Thomas algorithm**, a specialized Gaussian elimination for tridiagonal matrices: one forward sweep to eliminate the sub-diagonal, one backward sweep to solve.

The barrier as a boundary condition. The grid is truncated exactly at the barrier level, and that edge is held at $V = 0$ for every time step after expiry — physically, once the barrier is touched, the option is worthless forever after, and there is no need to track what happens on the other side of that wall at all.

Connects to code: `models/barrier_pde.py` — see §4.7.

2.12 In-Out Parity for Barrier Options

Definition. A **knock-in** option (activates only if a barrier is touched) and a **knock-out** option (deactivates if touched) with the same barrier and strike are complementary: between them, they cover every possible path, so their combined payoff is identical to a **vanilla** option with no barrier at all.

$$\textit{knock-in} + \textit{knock-out} = \textit{vanilla} \iff \textit{knock-in} = \textit{vanilla} - \textit{knock-out}$$

Why this is valuable in code. Instead of writing a second PDE solve for knock-in options (a different, more awkward boundary condition), the project prices vanilla analytically (Black-76) and subtracts the knock-out PDE price. This also means the parity check in `tests/test_barrier_pde.py` holds to floating-point precision **by construction**, not just approximately — see §7.

2.13 Scenario Analysis and the Greeks-Taylor Connection

Definition. Scenario analysis reprices an instrument under a grid of hypothetical market moves (the shocks) and reports the P&L versus today's price. It answers "what happens to my position if oil moves 10%?" directly, by full repricing, rather than by a linear approximation.

Connection to the Greeks. A Taylor expansion of price around today's F explains **why** a small shock's P&L should roughly match delta and gamma:

$$\Delta V \approx \Delta \cdot \Delta F + \frac{1}{2} \cdot \Gamma \cdot (\Delta F)^2$$

first two terms of a Taylor expansion in F

For a **small** shock (e.g. $F \pm 1\%$), the linear (delta) term dominates and the gamma term is a small, computable correction — this is exactly what `tests/test_scenario_analysis.py` checks (§7), and it is the theoretical reason the dashboard's delta-gamma chart (§4.8) tracks the actual reprice closely near the money and diverges further away.

Section 2 — What You Must Remember

- **Driftless GBM** ($dF = \sigma F dW$) is the one assumption every model shares — no drift term, ever.
- The **Black-76 PDE** has no first-derivative term; that absence is a direct consequence of the driftless assumption.
- **CRR** matches the diffusion via $\sqrt{\Delta t}$; **Leisen-Reimer** matches it via inverting the target probability directly (Peizer-Pratt) — that is the entire reason LR converges faster.
- **Monte Carlo** needs no discretization error for GBM (the log-step formula is exact) — only sampling error, which antithetic variates and the 95% CI address.
- **Crank-Nicolson** turns a PDE into a sequence of tridiagonal linear solves; the barrier is just a zero boundary condition on the grid.
- Common mistake: forgetting that vega/theta/rho need /100, /365, /100 scaling respectively to read as "per vol point / per day / per 1%" — the raw Greek is per **unit** change.

3 Codebase Structure

The complete project tree (test/cache noise omitted):

```
pricing_engine/
├── __init__.py           # exposes Black76, implied_vol at the top Level
├── README.md
├── example.py           # quick demo: python -m pricing_engine.example
├── validate_loz6.py     # validates Black-76 against real CME data
├── generate_loz6_figures.py # runs every visualization/ chart on real data
├── market_data/
│   ├── loader.py       # CSV -> MarketSlice (Phase 1)
│   └── loz6_settlements.csv # real CME WTI options settlement data
├── models/
│   ├── base.py         # OptionModel ABC + default FD Greeks
│   ├── black76.py      # analytic European pricer
│   ├── crr.py          # CRR binomial tree (American-capable)
│   ├── leisen_reimer.py # Leisen-Reimer binomial tree
│   ├── monte_carlo.py  # Asian / Lookback / Basket via simulation
│   └── barrier_pde.py  # Crank-Nicolson barrier option PDE
├── calibration/
│   └── implied_vol.py  # Newton-Raphson + bisection IV solver
├── greeks/
│   └── engine.py       # combined Greeks table / heatmap / report
├── scenario_analysis/
│   └── engine.py       # shock-grid repricer, single model or portfolio
├── visualization/
│   └── plots.py        # 9 matplotlib chart functions, saved as PNG
├── dashboard/
│   └── app.py          # Streamlit + Plotly interactive UI
├── tests/              # one pytest file per phase
│   ├── test_black76.py, test_crr.py, test_leisen_reimer.py
│   ├── test_monte_carlo.py, test_barrier_pde.py
│   └── test_greeks_engine.py, test_scenario_analysis.py
```

File / folder	Purpose	Why it exists	Connects to
market_data/loader.py	Parse a settlement CSV into typed dataclasses	Keeps file-format details out of the pricing code	Feeds F, T, r, and per-strike quotes into models/
models/base.py	Define the shared OptionModel contract + default Greeks	So every pricer is interchangeable everywhere else	Subclassed by all 6 pricers; used by greeks/, scenario_analysis/, visualization/
models/black76.py	Analytic closed-form European pricer	The fast, exact ground truth every other model is checked against	Used directly, and inside calibration/ and barrier_pde.py (vanilla + in-out parity)
models/crr.py	Binomial tree, American-capable	Handles early exercise, which Black-76 cannot	Cross-checked against black76.py in tests/
models/leisen_reimer.py	Faster-converging binomial tree	Same job as CRR, converges in far fewer steps	Compared against CRR in tests/ and the dashboard's convergence chart

models/monte_carlo.py	Path simulation for exotic payoffs	Asian/Lookback/Basket have no simple closed form	geometric_asian_price() is the analytic anchor used in tests/
models/barrier_pde.py	Finite-difference PDE solver	Barrier boundary conditions fit a grid naturally	Calls black76.py directly for the in-out parity term
calibration/implied_vol.py	Invert price -> volatility	Market quotes are in price/vol terms, models need sigma	Used by validate_loz6.py to check CME's own IV
greeks/engine.py	Aggregate Greeks across many models	One risk view regardless of which pricer produced each price	Consumes model.greeks(); feeds visualization/ and dashboard/
scenario_analysis/engine.py	Reprice under a shock grid	"What if F moves 10%" needs full repricing, not just Greeks	Consumes model._bumped(); feeds visualization/ and dashboard/
visualization/plots.py	Render matplotlib PNGs	A picture of a Greek/scenario is faster to read than a table	Wraps greeks/ and scenario_analysis/ outputs
dashboard/app.py	Interactive Streamlit + Plotly UI	Lets a user explore any model/instrument without writing code	Imports every layer below it; adds no new pricing logic
tests/	One pytest file per phase	Every model is validated against Black-76 or an independent check before being trusted	Exercises every module above

🔑 Section 3 — What You Must Remember

- `models/base.py` is the single most important file — read it before anything else.
- Every folder except `models/` and `market_data/` is **generic**: it takes any `OptionModel` and does something useful with it, without knowing which concrete pricer it received.
- `dashboard/app.py` is deliberately "dumb" — it contains no pricing math, only widget wiring.
- Common mistake: expecting `visualization/` or `scenario_analysis/` to import a **specific** model class — they only ever import `OptionModel`-shaped behaviour (`.price()`, `._bumped()`, `._u()`).

4 Code Walkthrough

This section goes through every major file. Snippets are excerpts, not full dumps — read alongside the actual source for full context.

4.1 models/base.py — The Shared Interface

Purpose: define what every pricer must provide, and give every pricer working Greeks for free.

```
@dataclass
class OptionModel(ABC):
    _underlying: str = "F"  # which field delta/gamma should bump

    @abstractmethod
    def price(self) -> float:
        raise NotImplementedError

    def _bumped(self, **changes) -> "OptionModel":
        return replace(self, **changes)  # dataclasses.replace: a modified copy
```

- **Inputs/Outputs:** `price()` takes nothing (all inputs are dataclass fields) and returns a `float`. `\_bumped(\*\*changes)` takes field overrides and returns a new instance — the original is never mutated.
- **Why a dataclass?** Free `\_\_init\_\_`, free `\_\_repr\_\_` (used as instrument labels in `greeks\_table()`), and `dataclasses.replace()` gives a one-line, type-safe way to create a bumped copy for Greeks/scenarios.
- **Why `\_underlying` as a field, not hard-coded "F"?** `BasketOption` later overrides what "the underlying" means (an average of several legs) without needing to override every Greek method.

base.py — the default finite-difference delta

```
def delta(self) -> float:
    u = self._u()
    h = max(u * 1e-4, 1e-6)
    up = self._bumped(**{self._underlying: u + h}).price()
    dn = self._bumped(**{self._underlying: u - h}).price()
    return (up - dn) / (2 * h)
```

- **Logic:** bump the underlying up and down by a tiny relative amount `h`, reprice both, take the central difference. `gamma/vega/theta/rho` follow the identical pattern on `F`, `sigma`, `T`, and `r` respectively.
- **Why central, not forward, differences?** Central differences are second-order accurate (error shrinks like `h<sup>2</sup>`) versus first-order for a forward difference (error shrinks like `h`) — much more accurate for the same `h`.
- **Dependencies:** none beyond `dataclasses` and `abc` — deliberately zero external dependencies, since every other module depends on this one.

4.2 models/black76.py — The Analytic Pricer

```
def _d1_d2(self):
    vsqrt = self.sigma * math.sqrt(self.T)
    d1 = (math.log(self.F / self.K) + 0.5 * self.sigma**2 * self.T) / vsqrt
    d2 = d1 - vsqrt
    return d1, d2

def price(self) -> float:
    disc = math.exp(-self.r * self.T)
    d1, d2 = self._d1_d2()
    if self.option_type == "call":
        return disc * (self.F * _norm_cdf(d1) - self.K * _norm_cdf(d2))
    return disc * (self.K * _norm_cdf(-d2) - self.F * _norm_cdf(-d1))
```

- **Purpose:** this is §2.3's formula, line for line. `_norm_cdf` is `0.5*(1+erf(x/v2))` — `math.erf` is the standard-library way to get the normal CDF without SciPy.
- **Why override the Greeks here instead of using the inherited FD version?** Speed and precision — the analytic derivatives are one-line formulas, exact to machine precision, and this class is the *reference* every other model is checked against, so it should not itself rely on an approximation.
- **Degenerate guard:** `if self.T <= 0 or self.sigma <= 0:` returns discounted intrinsic value directly — avoids a division by zero in `_d1_d2()` and gives the mathematically correct answer at expiry.

4.3 calibration/IMPLIED_vol.py — Solving for σ

```
sigma = sigma_init
for _ in range(max_iter):
    model = Black76(F=F, K=K, T=T, r=r, sigma=sigma, option_type=option_type)
    diff = model.price() - market_price
    if abs(diff) < tol:
        return sigma
    v = model.vega()
    if v < 1e-10:
        break
    sigma -= diff / v
```

- **Logic:** this is §2.5's Newton-Raphson update, using the model's own `vega()` as the derivative — no separate derivative code was written.
- **Fallback:** if vega collapses (deep in/out-of-the-money), the loop `break`'s and a bisection search over `sigma ∈ [1e-6, 5.0]` takes over, trading speed for guaranteed convergence.
- **Why it lives outside `models/`:** it is a *consumer* of `Black76`, not a pricer itself — it belongs in its own layer, matching §1.4's architecture.

4.4 models/crr.py — The CRR Binomial Tree

```
u = math.exp(self.sigma * math.sqrt(dt))
d = 1.0 / u
p = (1.0 - d) / (u - d)
```

```

disc = math.exp(-self.r * dt)

values = [self._intrinsic(self.F * u**(n-i) * d**i) for i in range(n+1)]

for step in range(n - 1, -1, -1):
    for i in range(step + 1):
        continuation = disc * (p * values[i] + (1-p) * values[i+1])
        if self.american:
            f_node = self.F * u**(step-i) * d**i
            values[i] = max(continuation, self._intrinsic(f_node))
        else:
            values[i] = continuation

```

- **Inputs:** same market fields as `Black76`, plus `n\_steps` (default 500) and `american: bool`.
Output: a single float, `values[0]` after the loop collapses the tree back to today.
- **Logic:** build every terminal payoff in one list comprehension, then walk backward layer by layer, overwriting `values` in place — no separate tree data structure is stored, which keeps memory at $O(n)$ instead of $O(n^2)$.
- **Why `american` is a plain boolean field, not a separate class:** European and American pricing differ by exactly one `max(...)` inside the loop — splitting them into two classes would duplicate the entire tree-building code.
- **Degenerate guard:** `if self.T <= 0 or self.sigma <= 0` returns intrinsic directly, because `u = e^0 = d = 1` would otherwise make `p = 0/0`.

4.5 models/leisen_reimer.py — The Leisen-Reimer Tree

```

def _peizer_pratt(z: float, n: int) -> float:
    denom = n + 1.0/3.0 + 0.1/(n + 1.0)
    inner = -((z / denom) ** 2) * (n + 1.0/6.0)
    sign = 1.0 if z >= 0 else -1.0
    return 0.5 + sign * math.sqrt(max(0.25 - 0.25 * math.exp(inner), 0.0))

p = _peizer_pratt(d2, n)
p_prime = _peizer_pratt(d1, n)
u = p_prime / p
d = (1.0 - p_prime) / (1.0 - p)

```

- This is §2.7's formula verbatim. Everything **after** computing `u, d, p` — the backward-induction loop — is textually identical to `crr.py`; only the parametrization of the tree's up/down/probability differs.
- **Why `n\_steps % 2 == 0: self.n\_steps += 1`** in `\_\_post\_init\_\_`: silently bumping an even count to the next odd one is friendlier than raising an error, since the caller's intent ("about this many steps") is preserved.
- **Dependencies:** `math` only — deliberately no NumPy, matching `crr.py`'s style.

4.6 models/monte_carlo.py — Simulation-Based Pricers

the shared exact-GBM path simulator (§2.2, §2.8)

```
def _simulate_paths(self) -> np.ndarray:
    dt = self.T / self.n_steps
    half = self.n_paths // 2 if self.antithetic else self.n_paths
    rng = np.random.default_rng(self.seed)
    z = rng.standard_normal((half, self.n_steps))
    if self.antithetic:
        z = np.concatenate([z, -z], axis=0) # mirror-image paths
    increments = -0.5*self.sigma**2*dt + self.sigma*math.sqrt(dt)*z
    log_paths = np.cumsum(increments, axis=1)
    return self.F * np.exp(log_paths)
```

- **Why `seed` is a dataclass field, not a local variable:** because `\_bumped()` copies all fields including `seed`, a delta bump and its unbumped counterpart draw *identical* random numbers ("common random numbers") — without this, Monte Carlo noise alone could produce a nonsensical negative gamma.
- **AsianOption.price()** averages `samples.mean(axis=1)` (arithmetic) or `exp(log(samples).mean(axis=1))` (geometric) over the sampled path, then applies `max(sign\*(avg-K), 0)`.
- **LookbackOption.price()** uses `terminal - paths.min(axis=1)` for a call — no `max(...,0)` needed, because the terminal price can never be below the path's own minimum (§2.9).
- **BasketOption.price()** correlates draws via `rng.standard\_normal(...) @ L.T` where `L = np.linalg.cholesky(corr)` — exactly §2.10's formula.

the analytic anchor used to validate the MC Asian option (§2.9.1)

```
def geometric_asian_price(F, K, T, r, sigma, n_steps, option_type="call"):
    n = n_steps
    mean_shift = -sigma**2 * T * (n + 1) / (4 * n)
    var_ln = sigma**2 * T * (n + 1) * (2*n + 1) / (6 * n * n)
    sigma_eff = math.sqrt(var_ln / T)
    F_eff = F * math.exp(mean_shift + 0.5 * var_ln)
    return Black76(F=F_eff, K=K, T=T, r=r, sigma=sigma_eff,
option_type=option_type).price()
```

Why `BasketOption` overrides `\_bumped()` and `vega()`: its `F` and `sigma` fields are per-leg *tuples*, not scalars, so the inherited `base.py` code (`self.sigma + h`) would crash. The override detects a tuple and applies a uniform shift across every leg instead.

4.7 models/barrier_pde.py — Crank-Nicolson PDE Solver

one Crank-Nicolson time step, §2.11

```
lower = -alpha[1:n]
diag = 1.0 + 2.0*alpha[1:n] + beta
upper = -alpha[1:n]

for step in range(1, m + 1):
    tau = step * dt
    v_low = math.exp(-self.r*tau) * intrinsic(f_min) # exact boundary value
    v_high = 0.0 if is_up else math.exp(-self.r*tau) * intrinsic(f_max)

    rhs = alpha[1:n]*V[0:n-1] + (1-2*alpha[1:n]-beta)*V[1:n] + alpha[1:n]*V[2:n+1]
```

```

rhs[0] += alpha[1] * v_low      # fold the known boundary into the RHS
rhs[-1] += alpha[n-1] * v_high

V[1:n] = _thomas_solve(lower, diag, upper, rhs)
V[0], V[-1] = v_low, v_high

```

- **Logic:** ``alpha`/`beta`` are precomputed once (§2.11's ``½Δτ·a`` and ``½Δτ·r``); each loop iteration builds the explicit half of Crank-Nicolson (``rhs``), folds in the known boundary values, then solves the implicit half via ``_thomas_solve`` — a hand-written tridiagonal solver (no SciPy dependency).
- **Why `V=0` is imposed at the barrier from the very first step**, not just at expiry: the option is knocked out the instant the barrier is touched at *any* time before expiry, not only if it happens to end there.
- **Knock-in via parity, not a second PDE:** ``price()`` for a ``*-in`` type calls ``Black76(...).price()`` and subtracts ``self._bumped(barrier_type=out_type)._knock_out_price()`` — exactly §2.12's formula.

4.8 greeks/engine.py — The Greeks Aggregator

```

def greeks_table(models: Sequence[OptionModel]) -> pd.DataFrame:
    records = [{"instrument": repr(m), **m.greeks()} for m in models]
    return pd.DataFrame.from_records(records, ...).set_index("instrument")

```

- **Why ``repr(m)`` as the row label:** every model is a dataclass, so Python auto-generates a full, unambiguous ``repr()`` — no per-model-type labeling code was needed.
- ``greeks_surface(model, strikes)`` reuses ``model._bumped(K=k)`` to sweep strikes — this is the data behind the heatmap and the dashboard's chart.
- ``sensitivity_report(models, positions)`` scales vega/theta/rho by /100, /365, /100 (§2.4's convention) and multiplies every column by position size, then sums a ``TOTAL`` row — a genuine portfolio rollup, not just a per-instrument dump.

4.9 scenario_analysis/engine.py — The Shock Grid

```

shocks = [("Base (0 shock)", lambda m: m._bumped())]
for pct in F_PCT_SHOCKS:
    for sign, sym in ((1, "+"), (-1, "-")):
        shocks.append((f"F {sym}{pct:.0%}",
                        lambda m, pct=_signed(pct, sign): m._bumped(F=_relative(m.F,
pct))))

```

- **Why ``pct=_signed(pct, sign)`` as a default argument:** capturing the loop variable directly in the lambda would be a classic Python closure bug — every lambda would see the *final* value of ``pct`/`sign`` after the loop finishes. Binding it as a default argument freezes the value at definition time.
- ``_relative`/`_absolute`` detect whether a field is a scalar or a per-leg tuple (basket) and shock it accordingly — the same generic pattern as ``base.py``'s Greeks.
- **Barrier/correlation shocks are added via ``hasattr(model, "barrier")`` / ``hasattr(model, "corr")``** — no ``isinstance`` checks against specific classes anywhere in this file.

- ``portfolio_scenario_table`` aligns every instrument to the **union** of shock names; a shock that does not apply to a given instrument contributes ``price=base, pnl=0`` rather than being dropped, so portfolio totals stay well-defined.

4.10 visualization/plots.py — Matplotlib Charts

```
def payoff_diagram(model, F_range=None, ...):
    u0 = model._u()
    Fs = np.linspace(*F_range or (0.5*u0, 1.5*u0), n)
    payoff = [model._bumped(F=f, T=0.0).price() for f in Fs]
```

- **Why `T=0.0`` gives the payoff, not a separate intrinsic-value function:** every model's own degenerate-time guard already returns intrinsic value at `T=0`` (§4.2, §4.4) — reusing it means the payoff diagram can never disagree with the model's own pricing logic.
- ``_u()`` instead of ``model.F`` throughout this file, so the same function works whether the underlying is a scalar (``Black76``) or a tuple (``BasketOption``) without branching on type.
- Backend is forced to ``Agg`` at import time (``matplotlib.use("Agg")``) — this module's job is to save PNGs, not open windows, so it must work headlessly.

4.11 dashboard/app.py — The Streamlit UI

```
def build_model():
    if category == "Barrier":
        return BarrierOption(F=F, K=K, T=T, r=r, sigma=sigma,
                             option_type=option_type,
                             barrier=barrier, barrier_type=barrier_type,
                             f_steps=f_steps_pde, t_steps=t_steps_pde)
    ...
```

- **Purpose:** translate sidebar widget values into one ``OptionModel`` instance, then call the exact same ``.price()``, ``.greeks()``, ``.greeks_table()``, ``.scenario_table()`` used everywhere else in the project.
- **No reimplemented pricing math anywhere in this file** — the only new logic is the Plotly rendering glue and, for barrier options specifically, a masked-intrinsic payoff diagram (since sweeping `F` past the barrier would otherwise violate ``BarrierOption``'s own validation in ``.__post_init__``).
- **The ``sys.path`` bootstrap at the top** (``Path(__file__).resolve().parents[2]``) exists because ``streamlit run`` only puts the script's own folder on ``sys.path``, not the caller's working directory — without it, ``import pricing_engine`` fails regardless of where the command is run from.

Section 4 — What You Must Remember

- Every pricer's core loop is short: ``black76.py`` is ~15 lines of formula, ``crr.py``/``leisen_reimer.py`` differ only in how ``u,d,p`` are computed, ``barrier_pde.py``'s loop is one Crank-Nicolson step repeated ``t_steps`` times.
- ``.bumped()`` and ``.u()`` are the two methods that let generic code (Greeks, scenarios, charts, dashboard) work across every model without type-checking.
- Common mistake: trying to bump a ``BasketOption``'s ``F``/``sigma`` the same way as a scalar model's — check ``models/monte_carlo.py``'s ``.bumped()``/``.vega()`` overrides first.

5 Theory → Code Connection

For every major concept: theory, then the formula, then exactly where it lives in the code, then the result it produces.

5.1 Black-76 Closed Form

Step	Content
THEORY	Driftless GBM + no-arbitrage PDE, solved analytically (§2.3)
FORMULA	$d1 = [\ln(F/K) + \frac{1}{2}\sigma^2 T] / (\sigma\sqrt{T})$; $\text{Call} = e^{-rT} [F \cdot N(d1) - K \cdot N(d2)]$
CODE	models/black76.py → <code>_d1_d2()</code> and <code>price()</code>
RESULT	$F=100, K=100, T=1, r=5\%, \sigma=25\% \rightarrow \text{price} = 9.4625$

5.2 Newton-Raphson Implied Volatility

Step	Content
THEORY	Invert $\text{price}(\sigma) = \text{market}$ by walking along the tangent line (§2.5)
FORMULA	$\sigma_{n+1} = \sigma_n - [V(\sigma_n) - \text{market}] / \text{vega}(\sigma_n)$
CODE	calibration/implied_vol.py → the for-loop in <code>implied_vol()</code>
RESULT	$\text{market} = 9.4625, \sigma_0 = 0.20 \rightarrow \text{converges to } \sigma = 0.2500$ in a handful of iterations

5.3 CRR Binomial Tree

Step	Content
THEORY	Discretize driftless GBM into up/down moves (§2.6)
FORMULA	$u = e^{(\sigma\sqrt{\Delta t})}$, $d = 1/u$, $p = (1-d)/(u-d)$
CODE	models/crr.py → <code>price()</code> , lines defining u, d, p
RESULT	500-step CRR call = 9.4634 vs. Black-76's 9.4625 (0.01% apart)

5.4 Peizer-Pratt / Leisen-Reimer

Step	Content
THEORY	Anchor the tree's terminal probability to the true $N(d_2)$ (§2.7)
FORMULA	$p = h(d_2, n)$, $p' = h(d_1, n)$, $u = p'/p$, $d = (1-p')/(1-p)$
CODE	models/leisen_reimer.py → <code>_peizer_pratt()</code> and <code>price()</code>
RESULT	101-step LR call = 9.4625, matching Black-76 to 4 decimal places (§7)

5.5 Exact GBM Simulation + Antithetic Variates

Step	Content
THEORY	Simulate the SDE's exact solution, mirror every draw (§2.2, §2.8)
FORMULA	$F(t+\Delta t) = F(t) \cdot \exp(-\frac{1}{2}\sigma^2\Delta t + \sigma\sqrt{\Delta t}Z)$, paired with $-Z$
CODE	models/monte_carlo.py → <code>_simulate_paths()</code>
RESULT	200,000-path European call = 9.50 ± 0.05 (95% CI contains Black-76's 9.4625)

5.6 Cholesky-Correlated Basket Simulation

Step	Content
THEORY	Factor the correlation matrix so independent draws become correlated (§2.10)
FORMULA	$\Sigma = L \cdot L^T$; $Z_{\text{corr}} = L \cdot Z$
CODE	models/monte_carlo.py → <code>BasketOption.price()</code> , the <code>np.linalg.cholesky(...)</code> line
RESULT	2-leg basket call ($\rho=0.5$) = 4.49, versus 5.02 at $\rho=0$ (higher correlation → more basket variance → higher call value)

5.7 Crank-Nicolson Barrier PDE

Step	Content
THEORY	Average explicit/implicit finite differences; barrier = zero boundary (§2.11)
FORMULA	Tridiagonal system per step, solved via the Thomas algorithm
CODE	models/barrier_pde.py → <code>_knock_out_price()</code> and <code>_thomas_solve()</code>
RESULT	Up-and-out call, barrier far away, converges to the vanilla price to within 0.1%

5.8 In-Out Parity

Step	Content
THEORY	knock-in + knock-out spans every path ⇒ equals the vanilla (§2.12)
FORMULA	knock-in = vanilla – knock-out
CODE	models/barrier_pde.py → <code>price()</code> , the "in" branch
RESULT	knock-in + knock-out reproduces the Black-76 price to 1e-9 (tests/test_barrier_pde.py)

5.9 Finite-Difference Greeks

Step	Content
THEORY	Approximate a derivative by repricing at $F \pm h$ (§2.4)
FORMULA	$\Delta \approx [V(F+h) - V(F-h)] / (2h)$
CODE	models/base.py → delta(), gamma(), vega(), theta(), rho()
RESULT	Matches Black-76's analytic delta to 4 decimal places for $h=1e-4 \cdot F$

Section 5 — What You Must Remember

- Every formula in §2 maps to a **specific, short** block of code — none of the pricers are more than ~60 lines of actual math.
- When two models disagree by more than a fraction of a percent for the same inputs, the bug is almost always in the newer model, since Black-76 is the trusted anchor.
- Common mistake: comparing a Monte Carlo price to an analytic price without checking the **confidence interval** first — a "mismatch" inside the CI is just sampling noise, not a bug.

6 Complete Execution Flow

Three concrete entry points, traced step by step.

6.1 python -m pricing_engine.validate_loz6

```
Input: hardcoded F, T, r + loz6_settlements.csv
→ load_slice(csv, F, T, r) [market_data/loader.py]
→ MarketSlice(F, T, r, quotes=[OptionQuote, ...])
→ for each quote, for each of (call, put):
    Black76(F, K=quote.strike, T, r, sigma=quote.cme_vol).price()
    implied_vol(quote.settle_price, F, K, T, r, option_type) [calibration/]
→ print a table comparing model price, solved IV, and CME's IV
→ print a put-call parity check (C-P vs. discounted F-K)
Output: a printed console table ($7 walks one row of this table by hand)
```

6.2 streamlit run pricing_engine/dashboard/app.py

```
Input: sidebar widget values (category, model, F,K,T,r,σ, barrier/basket extras)
→ build_model() [dashboard/app.py]
→ model.price(), model.confidence_interval() if MC
→ greeks_table([model]) [greeks/engine.py]
→ payoff sweep via model._bumped(F=f, T=0.0)
→ scenario_table(model) [scenario_analysis/engine.py]
→ CRR vs. Leisen-Reimer convergence sweep over n_steps
Output: live-updating price, Greeks table, Plotly payoff chart, scenario
        table, and convergence chart -- re-run top-to-bottom on every widget change
```

6.3 python -m pricing_engine.generate_loz6_figures

```
Input: same market data as validate_loz6.py
→ load_slice(...) → pick the ATM strike → Black76(...) model
→ for each of 9 functions in visualization/plots.py:
    generate(model, figures_dir="figures") → saves one PNG
Output: 9 PNG files in figures/ (payoff, P&L surface, Greeks heatmap,
        delta-gamma check, scenario bars, and the 4 Black-76 spec charts)
```

Section 6 — What You Must Remember

- All three entry points follow the same shape: **build market data** → **build a model** → **call price()/greeks()** → **hand the numbers to a presentation layer**.
- Streamlit re-runs the **entire script** on every interaction — there is no persistent state beyond what widgets remember, which is why `build_model()` is cheap to call repeatedly.
- Common mistake: expecting `generate_loz6_figures.py` or `validate_loz6.py` to be importable modules with reusable functions — they are top-to-bottom scripts, meant to be run, not imported.

7 Step-by-Step Example

One real instrument, traced through every layer of the engine: the **LOZ6 (December WTI crude) ATM call**, K=94, from `market\_data/loz6\_settlements.csv`, using CME's own quoted volatility.

7.1 Initial Input

Field	Value
F (futures price)	93.90
K (strike)	94.00
T (time to expiry)	68/365 = 0.186301 years
r (risk-free rate)	0.042 (4.2%)
σ (CME's quoted vol)	0.5058 (50.58%)
option_type	"call"

7.2 Black-76 Analytic Price

`Black76(F=93.9, K=94.0, T=0.1863, r=0.042, sigma=0.5058, option_type="call").price()` computes:

```
d1 = (ln(93.9/94.0) + 0.5*0.5058**2*0.1863) / (0.5058*0.1863**0.5)
    = 0.10428
d2 = d1 - 0.5058*0.1863**0.5 = -0.11403

N(d1) = 0.54153      N(d2) = 0.45461

price = e^(-0.042*0.1863) * (93.9*0.54153 - 94.0*0.45461)
       = 0.99219 * (50.826 - 42.733) = 8.05326
```

price = 8.0533 — matching the printed CME comparison from `validate\_loz6.py` (which shows the market settle at 8.07, a 0.02 gap, well within normal bid/ask noise).

7.3 Greeks (analytic)

Greek	Raw value	Per-convention value
Delta	0.5373	0.5373 (no scaling)
Gamma	0.01920	0.01920 (no scaling)
Vega	15.956	0.1596 per 1 vol point ($\div 100$)
Theta	-21.322	-0.0584 per day ($\div 365$)
Rho	-1.5003	-0.0150 per 1% rate move ($\div 100$)

7.4 Cross-Checking with CRR and Leisen-Reimer

Same inputs, European (`american=False`), at increasing step counts — this is exactly the dashboard's convergence chart (§4.11), reproduced here with real numbers:

n_steps	CRR price	CRR error	LR price	LR error
11	8.2377	0.1844	8.0503	0.0030

51	8.0927	0.0394	8.0531	0.0002
101	8.0731	0.0198	8.0532	0.00004
501	8.0572	0.0039	8.0533	~0.000001

At 101 steps, Leisen-Reimer is already accurate to 4 decimal places, while CRR still has a visible error at 501 steps — a direct, numerical demonstration of §2.7's $O(1/n^2)$ vs. $O(1/n)$ convergence claim.

7.5 Cross-Checking with Monte Carlo

`AsianOption(..., n_steps=1, n_paths=200_000, seed=7)` collapses the Asian average to a single point, i.e. a plain European payoff:

$$\text{price} = 8.0292 \quad 95\% \text{ CI} = [7.9702, 8.0883]$$

The Black-76 price (8.0533) sits comfortably inside this interval — the two methods agree once Monte Carlo's sampling error is accounted for.

7.6 Scenario Shock: F +1%

`scenario_table(model)` bumps F by +1% ($\Delta F = 93.9 \times 0.01 = 0.939$) and reprices:

```
new_price = 8.5662      pnl = 8.5662 - 8.0533 = 0.5130

# delta-only estimate (Taylor's 1st-order term):
delta * dF = 0.5373 * 0.939 = 0.5045

# residual, i.e. the curvature (gamma) contribution:
0.5130 - 0.5045 = 0.0084
0.5 * gamma * dF**2 = 0.5 * 0.01920 * 0.939**2 = 0.0085 # matches almost exactly
```

This is §2.13's Taylor expansion showing up in real numbers: the delta term explains ~98% of the P&L, and the gamma term explains almost all of the small remainder — exactly what `tests/test_scenario_analysis.py` asserts for every model in the engine.

Section 7 — What You Must Remember

- One instrument, five different views (analytic, two trees, Monte Carlo, scenario shock) — all agreeing within their respective error bars.
- The gap between a raw Greek and its "per-convention" reading ($\div 100$, $\div 365$, $\div 100$) is a units conversion, not a different number.
- Common mistake: comparing CRR at a *low* step count against Black-76 and concluding the tree is buggy — check the convergence trend (§7.4) before suspecting a bug.

8 Important Design Decisions

8.1 Why an Abstract Base Class + Dataclasses

- **Chosen:** `OptionModel(ABC)` as a `@dataclass`, subclassed by all 6 pricers.
- **Alternative considered:** plain functions (`black76_price(F,K,T,r,sigma,...)`) per model, with a separate Greeks module. Rejected because Greeks, scenario shocks, and the dashboard would all need per-model branching instead of one polymorphic call.
- **Trade-off:** dataclasses make every field public and mutable-by-replace, which is less strict than a fully encapsulated class — acceptable here since these objects are simple, short-lived value objects, not long-lived stateful services.

8.2 Why Finite-Difference Greeks by Default

- **Chosen:** `base.py` provides central finite-difference Greeks; only `Black76` overrides them analytically.
- **Why not require every model to derive analytic Greeks?** Trees, Monte Carlo, and PDE grids often have **no** clean closed-form derivative — finite differences are the only generic option, and they are "free" once `price()` exists.
- **Limitation observed in practice:** grid/tree-based gamma can come back as *exactly* 0.0 (seen with `BarrierOption`) when the FD bump size is smaller than the grid's own spacing — the bump falls inside one linear-interpolation cell and the curvature is invisible. This is a known trade-off, not a bug, and was intentionally left alone rather than hand-tuning the bump size per model.

8.3 Why a Fixed Random Seed as a Dataclass Field

- **Chosen:** every Monte Carlo model has a `seed: int` field, defaulted but explicit.
- **Why:** `_bumped()` copies all fields, so a Greek's "up" and "down" reprices draw *identical* random numbers (common random numbers) — this is what keeps Monte Carlo Greeks from being swamped by sampling noise, and it is also what makes `tests/test_greeks_engine.py`'s exact-equality check (rather than an approximate one) valid for a Monte Carlo model.
- **Alternative rejected:** reseeding from system entropy on every call — would make every price call non-reproducible and every Greek noisy.

8.4 Why In-Out Parity Instead of a Second PDE Solve

- **Chosen:** knock-in price = analytic vanilla – knock-out PDE price.
- **Alternative considered:** a separate PDE boundary condition for knock-in options. Rejected as unnecessary complexity — parity gives an **exact** relationship for free, and it was already needed as a correctness check.
- **Trade-off:** the knock-in price inherits whatever numerical error the knock-out PDE has. For deep-out-of-the-money, far-barrier cases this can occasionally produce a tiny negative price (observed during Phase 7 testing, on the order of -0.01 against a price near zero) — a numerical artifact of the grid, not a sign error.

8.5 Library Choices

Library	Used for	Why chosen
NumPy	Vectorized Monte Carlo paths, Cholesky decomposition, PDE grids	The standard for numerical arrays in Python; no simulation of this scale is practical in pure Python loops
pandas	Greeks tables, scenario tables	DataFrames are the natural shape for "one row per instrument/shock"
Matplotlib	Static PNG charts (visualization/)	Ubiquitous, headless-friendly (Agg backend), no server needed
Plotly	Interactive dashboard charts	Explicitly requested for the dashboard's hover/zoom interactivity
Streamlit	The dashboard itself	Turns a Python script into a web UI with no separate frontend code
statistics.NormalDist	The 95% CI z-value, geometric Asian formula	Exact standard-library normal quantiles — no need for SciPy just for this
math.erf	The normal CDF inside Black-76	Standard-library; avoids a SciPy dependency for one function

Notably not used: SciPy (available in the environment, but every numerical solve — the barrier PDE's tridiagonal system, the implied-vol root-find — was hand-written instead of calling ``scipy.linalg.solve_banded`` or ``scipy.optimize.brentq``). This was a deliberate choice to keep the numerical methods themselves visible and teachable in the codebase, at the cost of a little extra code versus a one-line library call.

8.6 Known Limitations

- **Uniform PDE grid resolution degrades for a very distant barrier:** discovered directly while testing Phase 7 — a barrier set unrealistically far away (e.g. 1,000,000 vs. a spot of 100) under-resolves the region near the strike with a fixed step count, giving a wildly wrong price. The fix was to test with a **realistic** far barrier (several σ away) rather than an absurd one; a production system would use an adaptive or non-uniform grid instead.
- **No true 2D PDE for basket options** — baskets are priced by Monte Carlo only; a 2-asset PDE grid would be more accurate for small baskets but does not scale past 2-3 assets, which Monte Carlo handles uniformly.
- **No dividend yield / cost-of-carry parameter** — every model assumes the pure Black-76 driftless case; pricing an option on a dividend-paying stock (nonzero cost of carry) would need an extra field threaded through every model.
- **The dashboard recomputes everything on every widget change** — acceptable at the Monte Carlo path counts used (20,000 by default) but would need caching for a much heavier workload.

8.7 Possible Improvements

- Rannacher smoothing (a few implicit Euler steps at the start of the Crank-Nicolson scheme) to damp the small oscillations near a payoff kink.
- A non-uniform (e.g. log-spaced or barrier-concentrated) PDE grid so a distant barrier no longer degrades resolution near the strike.
- Control-variate variance reduction for the Monte Carlo Asian option, using the geometric Asian closed form (§2.9.1) as the control — the two payoffs are highly correlated, so this would cut required paths substantially.
- A real closed-form (Reiner-Rubinstein-style) barrier formula as a second cross-check for the PDE, the same role Black-76 already plays for the trees.

Section 8 — What You Must Remember

- Almost every design choice traces back to one goal: **make Greeks, scenarios, and visualization work for free on any new model.**
- The two genuine numerical limitations found during development (far-barrier grid resolution, small-value knock-in noise) were **diagnosed with independent Monte Carlo cross-checks**, not assumed away — that habit is worth keeping when extending this project.
- Common mistake: treating a Monte Carlo vs. analytic mismatch as a bug before checking whether it is simply outside the confidence interval.

9 Learning Notes (Consolidated)

Sections 1–8 each closed with a **Key Takeaway** box scoped to that section. This section pulls the single most important formula and function from *every* section into one place, for quick review before the final cheat sheet.

9.1 One Formula per Section

Section	The one formula to remember
1. Overview	n/a — architecture, not math
2. Theory	$d1 = [\ln(F/K) + \frac{1}{2}\sigma^2 T] / (\sigma\sqrt{T})$ — everything else is built around this
3. Structure	n/a — organization, not math
4. Walkthrough	$\Delta \approx [V(F+h) - V(F-h)] / (2h)$ — the one idea every non-analytic model relies on
5. Theory ↔ Code	knock-in = vanilla – knock-out — the cleanest theory-to-code shortcut in the project
6. Execution Flow	n/a — process, not math
7. Example	$\Delta V \approx \Delta \cdot \Delta F + \frac{1}{2} \cdot \Gamma \cdot (\Delta F)^2$ — verified numerically end to end
8. Design	n/a — engineering trade-offs, not math

9.2 One Function/Class per Module

Module	The one thing to know how to call
models/base.py	OptionModel._bumped(**changes) — a modified copy for any Greek/scenario
models/black76.py	Black76(F, K, T, r, sigma, option_type).price()
models/crr.py / leisen_reimer.py	CRRBinomial(..., n_steps, american=True/False).price()
models/monte_carlo.py	AsianOption/LookbackOption/BasketOption(..., n_paths, seed).price() + .confidence_interval()
models/barrier_pde.py	BarrierOption(..., barrier, barrier_type).price()
calibration/IMPLIED_VOL.py	IMPLIED_VOL(market_price, F, K, T, r, option_type)
greeks/engine.py	greeks_table(list_of_models)
scenario_analysis/engine.py	scenario_table(model)
visualization/plots.py	payoff_diagram(model), theta_decay(model), ... (9 functions, same signature shape)
dashboard/app.py	no function to call — run with `streamlit run dashboard/app.py`

9.3 Common Mistakes, Collected

- Forgetting the $\div 100$ / $\div 365$ / $\div 100$ convention for vega/theta/rho and misreading a raw Greek as if it were already per-day or per-vol-point.
- Judging a tree (CRR/LR) as "wrong" from a single low step count instead of checking the convergence trend.

- Judging a Monte Carlo price as "wrong" without checking whether the analytic price falls inside its 95% CI.
- Trying to bump a `BasketOption`'s`F`/`sigma`` (tuples) the same way as a scalar model's field.
- Running the Streamlit dashboard from a directory where `pricing_engine`` is not importable (fixed by the `sys.path`` bootstrap in `dashboard/app.py``, §4.11).

9.4 What You Must Understand Before Modifying This Codebase

11. How `OptionModel._bumped()`` and `._u()`` work — every generic tool in the project is built on these two methods.
12. That every pricer's `price()`` is the **only** required method — Greeks are free unless you choose to override them for speed/accuracy.
13. The driftless assumption (`dF=σFdW``, no drift term) — it is why every formula in this project looks slightly different from a textbook stock-option formula.
14. That Black-76 is the trusted reference every other model is checked against — when adding model #7, write a test that compares it to Black-76 (or, for exotics, to an independent closed form) before trusting it.

10 Final Cheat Sheet

10.1 Architecture in One Picture

- **Layer 1 — data:** ``market_data/`` reads a CSV into typed objects.
- **Layer 2 — pricing:** ``models/`` — one ABC (``OptionModel``), six concrete pricers, all sharing ``.price()/`.greeks()`.
- **Layer 3 — analytics:** ``calibration/`, `greeks/`, `scenario_analysis/`, `visualization/`` — generic tools over any ``OptionModel``.
- **Layer 4 — presentation:** ``dashboard/app.py`` — Streamlit + Plotly, zero pricing logic of its own.

10.2 Key Concepts

- **Driftless GBM:** ``dF = σF dW`` — no drift, because futures cost nothing to enter.
- **Black-76 PDE:** ``∂V/∂t + ½σ²F²V_FF - rV = 0`` — no first-derivative term.
- **Risk-neutral tree pricing:** discount the probability-weighted average of the two children nodes; American = also compare against immediate exercise.
- **Peizer-Pratt inversion:** pick tree probabilities to match the true terminal probability directly, instead of via ``√Δt`` — faster convergence.
- **Exact GBM simulation:** ``F(t+Δt)=F(t)·exp(-½σ²Δt+σ√Δt·Z)`` — no discretization bias.
- **Antithetic variates:** pair every draw ``Z`` with ``-Z`` for free variance reduction.
- **Cholesky correlation:** ``Σ=L·Lᵀ``; correlate independent draws via ``L·Z``.
- **Crank-Nicolson:** average explicit + implicit finite differences; barrier = a zero boundary condition on the grid.
- **In-out parity:** ``knock-in = vanilla - knock-out``.
- **Greeks-Taylor connection:** ``ΔV ≈ Δ·ΔF + ½Γ·(ΔF)²`` is **why** small scenario shocks track `delta/gamma`.

10.3 Formula Quick-Reference

Concept	Formula
Black-76 d1/d2	$d1 = [\ln(F/K) + \frac{1}{2}\sigma^2 T] / (\sigma\sqrt{T})$; $d2 = d1 - \sigma\sqrt{T}$
Black-76 call/put	Call = $e^{-rT}[F \cdot N(d1) - K \cdot N(d2)]$; Put = $e^{-rT}[K \cdot N(-d2) - F \cdot N(-d1)]$
FD delta/gamma	$\Delta \approx [V(F+h) - V(F-h)] / 2h$; $\Gamma \approx [V(F+h) - 2V(F) + V(F-h)] / h^2$
Newton-Raphson IV	$\sigma_{n+1} = \sigma_n - [V(\sigma_n) - \text{market}] / \text{vega}(\sigma_n)$
CRR	$u = e^{\sigma\sqrt{\Delta t}}$; $d = 1/u$; $p = (1-d)/(u-d)$
Peizer-Pratt	$p = h(d2, n)$; $p' = h(d1, n)$; $u = p'/p$; $d = (1-p)/(1-p)$
Exact GBM step	$F(t+\Delta t) = F(t) \exp(-\frac{1}{2}\sigma^2 \Delta t + \sigma\sqrt{\Delta t} \cdot Z)$
95% CI	$\text{mean} \pm 1.96 \cdot (\text{std} / \sqrt{N})$
Cholesky	$\Sigma = L \cdot L^T$; $Z_{\text{corr}} = L \cdot Z_{\text{independent}}$
Geometric Asian	$F_{\text{eff}} = F \cdot \exp(\text{mean_shift} + \frac{1}{2}\text{var_ln})$; $\sigma_{\text{eff}} = \sqrt{\text{var_ln}/T}$; price = Black76(F_{eff} , σ_{eff})
In-out parity	knock-in + knock-out = vanilla
Scenario Taylor check	$\Delta V \approx \Delta \cdot \Delta F + \frac{1}{2}\Gamma \cdot (\Delta F)^2$

10.4 Important Functions / Classes

Name	File
OptionModel, ._bumped(), ._u()	models/base.py
Black76	models/black76.py
implied_vol()	calibration/implied_vol.py
CRRBinomial	models/crr.py
LeisenReimer, _peizer_pratt()	models/leisen_reimer.py
AsianOption, LookbackOption, BasketOption, geometric_asian_price()	models/monte_carlo.py
BarrierOption, _thomas_solve()	models/barrier_pde.py
load_slice()	market_data/loader.py
greeks_table(), greeks_surface(), sensitivity_report()	greeks/engine.py
scenario_table(), portfolio_scenario_table()	scenario_analysis/engine.py
payoff_diagram(), pnl_surface(), theta_decay(), ...	visualization/plots.py
build_model()	dashboard/app.py

10.5 Execution Flow (Any Entry Point)

Market inputs → build a model (dataclass) → model.price()/greeks() → aggregate (Greeks table / scenario table) → present (print / PNG / dashboard)

10.6 Inputs / Outputs Summary

Model	Required inputs	Extra inputs	Output of price()
Black76	F,K,T,r,σ,option_type	—	float
CRRBinomial / LeisenReimer	F,K,T,r,σ,option_type	n_steps, american	float
AsianOption / LookbackOption	F,K,T,r,σ,option_type	n_paths, n_steps, seed, antithetic	float (+ .confidence_interval())
BasketOption	F(tuple),σ(tuple),corr,K,T,r,option_type	weights, n_paths, n_steps, seed	float (+ .confidence_interval())
BarrierOption	F,K,T,r,σ,option_type,barrier,barrier_type	f_steps, t_steps	float

10.7 Assumptions

- Underlying is a **driftless** GBM (a futures price, not a dividend/carry-bearing spot).
- Volatility and the risk-free rate are constant over the life of the option (no term structure).
- No transaction costs, no bid/ask spread, continuous hedging is implicitly assumed by the PDE derivation.
- Barrier monitoring is continuous (the PDE grid effectively checks every instant, not just daily closes).

10.8 Limitations

- Fixed, uniform grids (PDE and tree) lose accuracy if a barrier/strike is set very far from the current price without also increasing the step count.
- No dividend yield / cost-of-carry parameter anywhere in the engine.
- Basket options are Monte-Carlo-only; no PDE cross-check exists for them.
- The dashboard has no caching — every widget change reprices from scratch.



Final Takeaway

- This entire engine rests on **one interface** (`OptionModel``) and **one assumption** (driftless GBM). Once both are second nature, every file in this project reads as a variation on the same theme.
- When extending the project: implement `price()`, write a test comparing it to an existing trusted model, and everything else — Greeks, scenarios, charts, the dashboard — follows automatically.